Selection Sort Time Complexity

```
#include <iostream>
using namespace std;

void asc_selection_sort(int arr[], int size) {
   int temp, i, j, index_min;
   for (i = 0; i < size - 1; i++) {
      index_min = i;
      for (j = i + 1; j < size; j++) {
         if (arr[j] < arr[index_min]) {
            index_min = j;
         }
      }
      if (index_min != i) {
         swap(arr[i] , arr[index_min]);
      }
   }
}
void show(int arr[], int size) {
   for (int i = 0; i < size; i++) {
      cout << arr[i] << " ";
   }
   cout << endl;
}

int main() {

   int arr[5]={5,12,0,1,8};
   asc_selection_sort(arr, 5);
   show(arr, 5);

   return 0;
} ---> calculate  the time complexity of that selection sort using comparison calculation with iterative simulation
```

Time Complexity Calculation of Selection Sort Using Comparison Count

Step 1: Understanding Selection Sort

Selection sort works by repeatedly selecting the smallest element from the unsorted portion and swapping it with the first element of the unsorted portion.

For an array of size n, selection sort does the following:

1. In the **first pass** ($i = 0$), it compares $(n - 1)$ elements.

2. In the **second pass** ($i = 1$), it compares $(n - 2)$ elements.
3. In the **third pass** ($i = 2$), it compares $(n - 3)$ elements.
4. This continues until the second last element, which requires only one comparison.

Step 2: Number of Comparisons in Selection Sort

For an array of size n, the total number of comparisons is:

$$(n-1) + (n-2) + (n-3) + \cdots + 2 + 1$$

This forms an arithmetic series:

$$\sum_{i=1}^{n-1} i = \frac{(n-1) \cdot n}{2}$$

Thus, the number of comparisons is:

$$\frac{n(n-1)}{2}$$

For **n = 5** in the given code:

$$\frac{5(5-1)}{2} = \frac{5 \times 4}{2} = 10 \text{ comparisons}$$

Step 3: Big-O Notation

- The dominant term in $\frac{n(n-1)}{2}$ is n^2.
- Dropping the constant factors, the **time complexity** of selection sort in terms of comparisons is:

$$O(n^2)$$

Step 4: Iterative Simulation (Manual Comparison Count)

For the given array:
Initial Array: $5, 12, 0, 1, 8$

Pass	Comparisons
1st (i=0)	4 (compare 5 with 12, 0, 1, 8)
2nd (i=1)	3 (compare 12 with 1, 8)
3rd (i=2)	2 (compare 12 with 8)
4th (i=3)	1 (compare 12 with 8)
Total	**10 Comparisons**

Thus, this confirms our formula-based calculation.